

Informe Técnico: Pipeline de Datos Integrador - Arquitectura Medallion

Diplomatura en Ingeniería de Datos

Estudiante: Jonathan Constante - Técnico Analista de Sistema

Profesor : Gutierrez Antonio Martín

Fecha: 11-05-2026

1. Introducción y Objetivo

El presente proyecto tiene como objetivo diseñar e implementar un pipeline de datos de punta a punta utilizando la plataforma Azure. El problema de negocio resuelto consiste en la unificación de datos de ventas diarias con un catálogo de productos externos para generar reportes de rentabilidad por categoría.

Se busca automatizar la ingesta y transformación de datos para que el nivel gerencial pueda visualizar, de forma clara, cuáles son los productos y categorías con mayor impacto en la facturación.

2. Tecnologías Utilizadas

Para el desarrollo del pipeline de datos se utilizaron las siguientes tecnologías y servicios cloud:

- Microsoft Azure como plataforma principal de infraestructura y servicios.
- Azure Data Factory para la orquestación y automatización de pipelines.
- Azure Databricks para el procesamiento distribuido y transformación de datos mediante notebooks.
- Apache Spark / PySpark para el procesamiento de grandes volúmenes de datos.
- Azure Blob Storage como almacenamiento de datos en la capa Bronze.
- Delta Lake para almacenamiento optimizado y procesamiento transaccional.
- Archivos CSV como fuente de datos transaccionales.
- Archivos JSON como fuente de datos semiestructurados.
- Triggers programados para automatización de ejecuciones diarias.

3. Fuentes de Datos (Heterogéneas)

Para cumplir con la consigna de fuentes de datos diversas, se utilizaron:

- **Ventas Diarias (CSV):** Archivo plano que contiene el detalle de transacciones, IDs de productos y cantidades.
- **Catálogo de Productos (JSON):** Estructura jerárquica obtenida de una simulación de API que incluye nombres (títulos), precios unitarios y categorías.

4. Arquitectura General del Pipeline

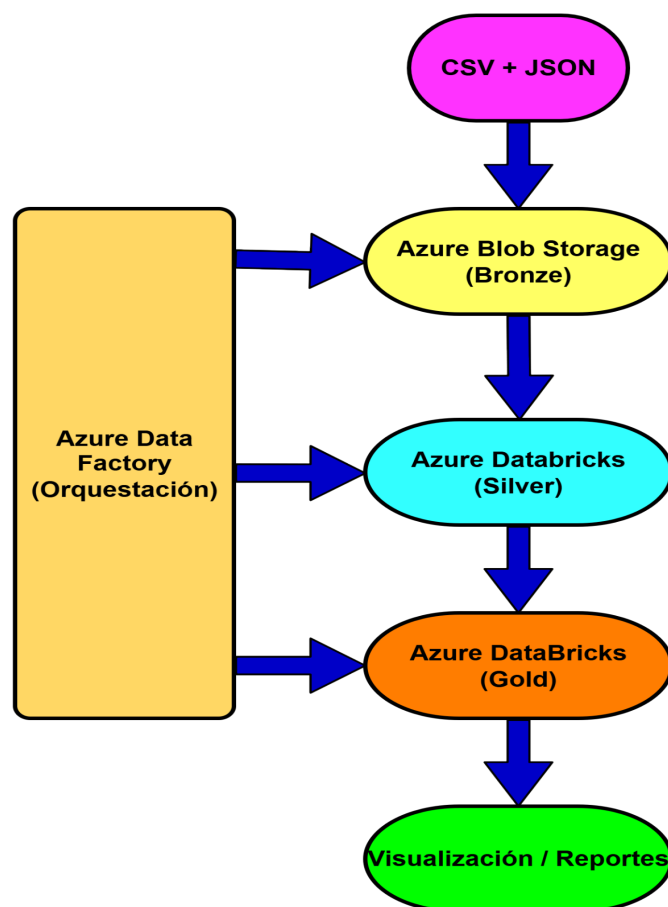


Figura 1 - Arquitectura General del Pipeline

Descripción de la Arquitectura

La siguiente arquitectura representa el flujo completo de procesamiento implementado para el proyecto integrador. El pipeline inicia con la ingesta de datos provenientes de fuentes heterogéneas en formato CSV y JSON, los cuales son almacenados inicialmente en Azure Blob Storage dentro de la capa Bronze.

Posteriormente, mediante notebooks ejecutados en Azure Databricks, los datos son transformados y validados en la capa Silver, aplicando reglas de limpieza, normalización y enriquecimiento. Finalmente, los datos consolidados son procesados en la capa Gold para generar estructuras analíticas optimizadas para reporting y visualización.

La orquestación integral del flujo fue realizada mediante Azure Data Factory, utilizando pipelines automatizados y triggers programados para garantizar la ejecución periódica del proceso ETL.

5. Arquitectura Medallion

Se implementó un flujo de datos dividido en tres capas lógicas para asegurar la trazabilidad y calidad:

● Capa Bronze (Raw)

- **Proceso:** Ingesta de los archivos CSV y JSON desde el origen hacia contenedores de Azure Blob Storage sin modificaciones.
- **Objetivo:** Mantener una copia fiel de los datos crudos para posibles reprocesamientos.

● Capa Silver (Limpieza y Enriquecimiento)

Para la transición de los datos desde la capa Bronze a la capa Silver, se desarrolló el Notebook principal de procesamiento titulado Transformaciones_Ventas.

En este componente se aplicaron las reglas de negocio críticas, tales como:

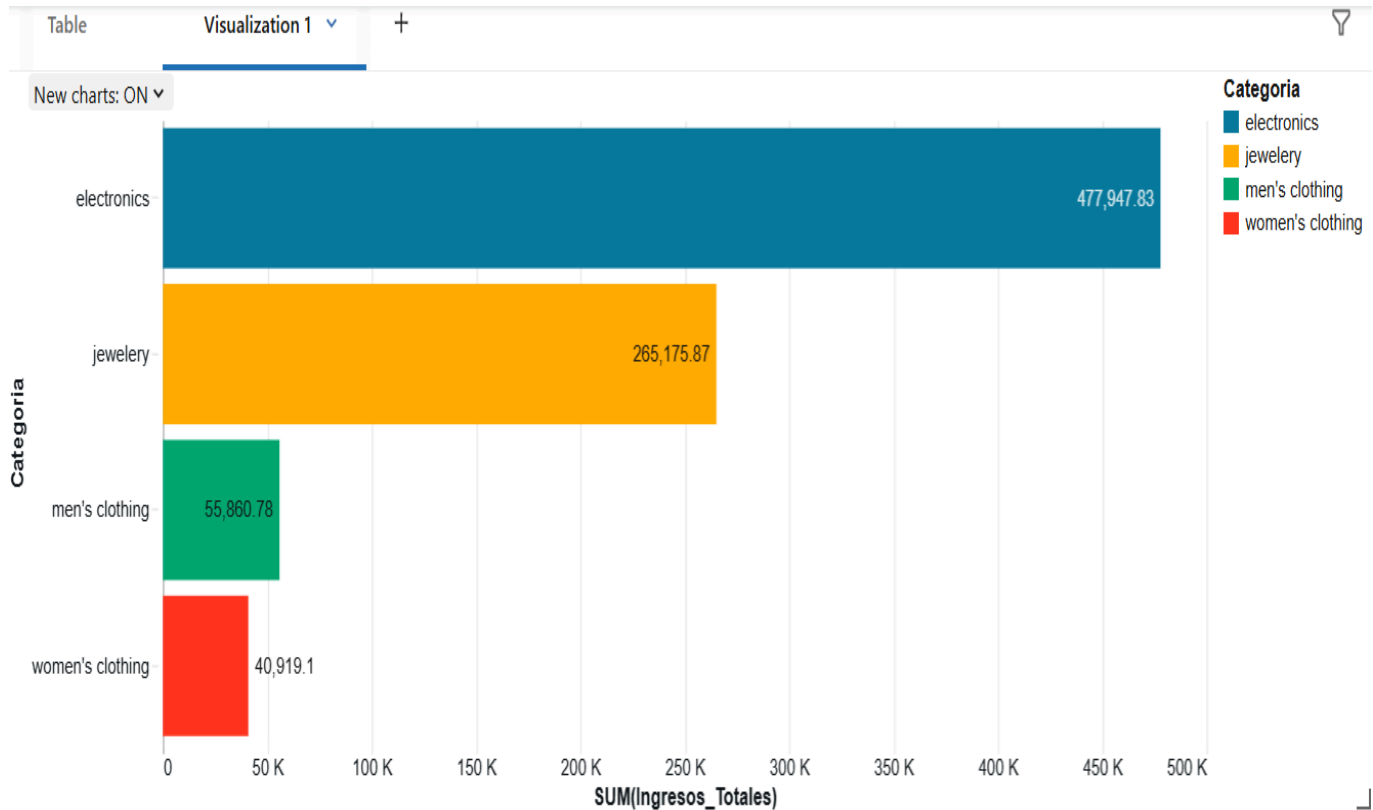
- Normalización de nombres de columnas.
- Conversión de tipos de datos (específicamente formatos de fecha y moneda).
- Filtro de registros nulos o duplicados para asegurar la integridad de la información antes de su paso a la capa analítica (Gold).
- Se eliminaron los registros con IDs de producto nulos. Se realizó un **Join** entre ambas fuentes para consolidar la información.

● Capa Gold (Analítica)

- **Proceso:** Creación de tablas de agregación. Se calculó la métrica **Venta_Total** (Cantidad * Precio) y se agrupó por **Categoría**.
- **Resultado:** Una tabla optimizada para lectura gerencial con un gráfico de barras que identifica el ranking de ingresos.

Figura del Gráfico Final de Databricks con las barras por categoría

Figura 2 - Visualización Analítica de Ventas por Categoría



Se realizó una visualización en la capa Gold para identificar rápidamente el flujo de ingresos por categoría. Esta agregación demuestra la correcta integración de las fuentes heterogéneas y la aplicación de las reglas de negocio sobre el volumen total de transacciones.

6. Orquestación y Automatización

La orquestación completa del flujo se realizó mediante Azure Data Factory (ADF).

- **Pipeline:** Se encadenaron tres actividades de tipo "Databricks Notebook" (Bronze -> Silver -> Gold).
- **Automatización (Trigger):** Se configuró un disparador de tipo *Schedule* para que el pipeline se ejecute automáticamente todos los días a las [08:00 AM], garantizando datos actualizados cada mañana.

Figura del Pipeline de ADF con las flechas verdes y el icono del reloj del Trigger

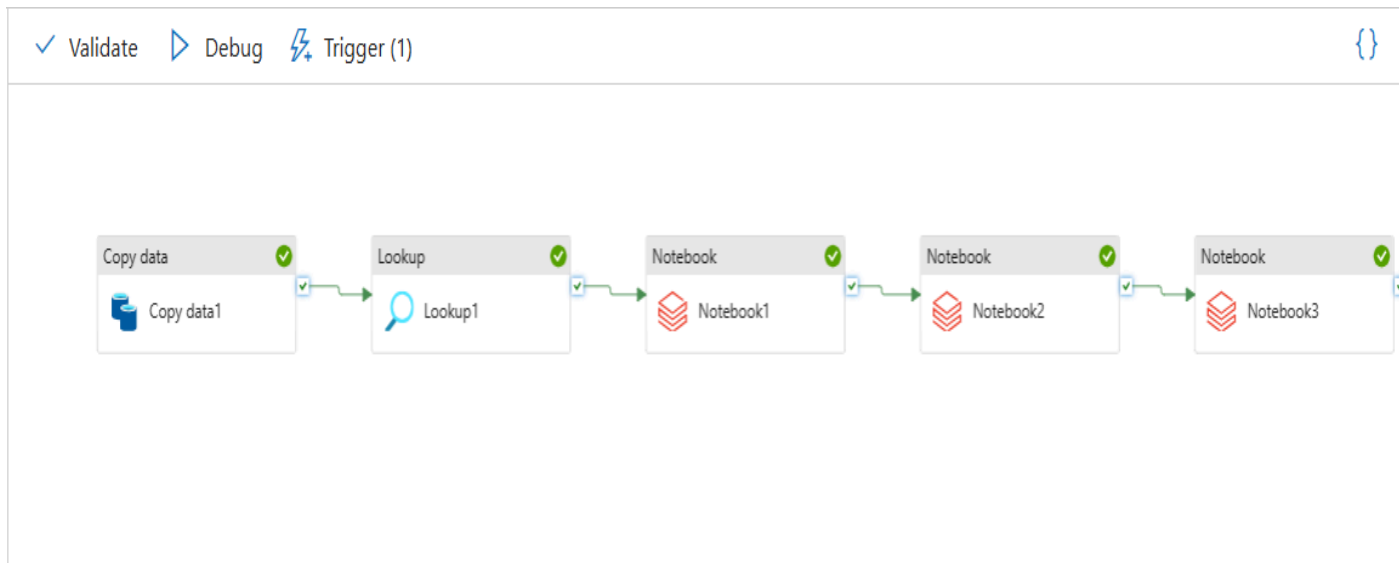


Figura 3 - Pipeline Orquestado en Azure Data Factory

7. Criterios de Calidad y Manejo de Errores

Se implementaron reglas de validación explícitas en el código de PySpark:

1. **Validación de Nulos:** Se filtraron registros donde el campo `ID_Producto` era inexistente para evitar errores en el Join.
2. **Validación de Tipos:** Conversión forzada de strings a numéricos para cálculos matemáticos.
3. **Tratamiento de Errores:** Los registros inválidos fueron contabilizados y descartados del flujo principal para no afectar la integridad de la capa Gold.

8. Control de Calidad y Métricas de la Capa Gold

Para este proyecto, se tomó como referencia principal la entidad `tabla_productos_final`, ubicada en el esquema `default`, la cual consolida la información procesada y enriquecida proveniente de las capas Bronze y Silver.

A continuación, se presentan las principales métricas obtenidas durante la ejecución del pipeline:

Métricas de Control	Métrica	Resultado
Registros Consolidados en Gold	20	tabla_productos_final
Registros Descartados / Errores	0	log_errores_ventas
Validación de Integridad de Datos	100%	validación de esquema Spark
Facturación Total Procesada	\$ 3240,92	Columna price (Capa Gold)
Categorías Únicas Detectadas	4	Columna category (Capa Gold)

Estas métricas permiten validar la consistencia, integridad y calidad de la información generada por el pipeline, asegurando que los datos analíticos disponibles en la capa Gold sean confiables para procesos de reporting y toma de decisiones.

9. Fragmentos de Código PySpark (Implementación)

9.1. Capa Bronze: Ingesta y Conexión

Figura 4: Código de Ingesta y Conexión

```
# 1. Parámetros (Widgets)
dbutils.widgets.text("storage_name", "")
dbutils.widgets.text("container_name", "bronze")

v_storage = dbutils.widgets.get("storage_name")
v_container = dbutils.widgets.get("container_name")

# 2. Configuración de la conexión
spark.conf.set(
    f"fs.azure.account.key.{v_storage}.dfs.core.windows.net",
    "xyyl02JGjOm0Nzsra0AXhEtVqSNnbxJTYeq/AT0eYbtHTsvnNEZp5DjOr+IzVpM2zG9vOXGffmX7+AStroxYSw=="
)
```

```
# Definimos la ruta de entrada
path_entrada = f"abfss://{v_container}@{v_storage}.dfs.core.windows.net"

# Leemos el JSON
df_productos = spark.read.json(f"{path_entrada}/productos.json")
```

En esta etapa se parametriza la conexión con el Azure Data Lake Storage Gen2 utilizando claves de acceso. Se realiza la lectura del archivo crudo en formato JSON. Esta capa es de solo lectura y mantiene el formato original de la fuente.

9.2. Capa Silver: Validación y Calidad de Datos

Figura 5: Código Validación y Calidad de Datos

```
from pyspark.sql.functions import col

# 1. Filtramos los registros que tienen ID (Válidos)
df_silver = df_productos.filter(col("id").isNotNull())

# 2. Filtramos los que NO tienen ID (Inválidos)
df_invalidos = df_productos.filter(col("id").isNull())

# 3. Guardamos los inválidos en una tabla Delta de LOGS
df_invalidos.write.format("delta").mode("append").saveAsTable("rejected_products_logs")
```

Se aplica la lógica de limpieza. Se filtran los registros nulos para asegurar la integridad de la información. Los datos que no cumplen las reglas de calidad se derivan a una tabla de auditoría (rejected_products_logs), mientras que los datos limpios avanzan en el pipeline.

9.3. Capa Gold: Persistencia y Entrega de Negocio

Figura 6: Código de persistencia y Entrega de Negocio

```
# Definimos la ruta de salida
path_salida = f"abfss://{silver@{v_storage}}.dfs.core.windows.net/productos_procesados"

# Guardamos como tabla Delta física
df_silver.write.format("delta").mode("overwrite").save(path_salida)

# Registramos la tabla en el catálogo para consultas SQL
df_silver.write.format("delta").mode("overwrite").saveAsTable("tabla_productos_final")

print(";Pipeline completado! Los datos limpios están en la capa Silver/Gold.")
```

En esta etapa final, los datos ya procesados se transforman al formato optimizado Delta Lake. La información se registra en el Metastore de Databricks como `tabla_productos_final`, quedando disponible para el consumo de analítica avanzada y herramientas de BI, garantizando una "Única Fuente de Verdad".

10. Implementación: Notebook Transformaciones_Ventas (Capa Silver)

Este notebook realiza la limpieza y estandarización de las fuentes de datos, consolidando archivos de diferentes orígenes en tablas con formato Delta.

10.1. Configuración y Orquestación de Ingesta

Figura 7: Código Configuración y Orquestación de Ingesta

```
# 1. CONFIGURACIÓN Y PARÁMETROS
dbutils.widgets.text("storage_name", "datalaketpint1")
v_storage = dbutils.widgets.get("storage_name")

# Configuración de acceso al Storage
spark.conf.set(
    f"fs.azure.account.key.{v_storage}.dfs.core.windows.net",
    "xyy102JGj0m0Nzsra0AXhEtVqSNnbxJTYeq/AT0eYbtHTsvnNEZp5Dj0r+IzVpM2zG9v0XGffmX7+AStroxYSw=="
)
```

```
from pyspark.sql.functions import col

# 2. PROCESAR VENTAS (CSV)
path_ventas = f"abfss://{v_storage}.dfs.core.windows.net/ventas.csv"

df_ventas = spark.read.format("csv") \
    .option("header", "True") \
    .option("inferSchema", "True") \
    .load(path_ventas)
```

En este primer bloque se realiza la orquestación técnica del notebook. Se utilizan Widgets para parametrizar el nombre del Storage Account, permitiendo que el código sea reutilizable en distintos entornos. Además, se configura la seguridad mediante la clave de acceso de Azure y se realiza la lectura del archivo `ventas.csv`

en formato crudo, utilizando inferSchema para que PySpark identifique automáticamente los tipos de datos de las columnas.

10.2. Limpieza de Ventas y Calidad de Datos

Figura 8: Limpieza de Ventas y Calidad de Datos

```
# Limpieza de Ventas
df_ventas_ok = df_ventas.filter(
    (col("ID_Venta").isNotNull()) & (col("Cantidad") > 0)
)

# Guardar Ventas en Silver
df_ventas_ok.write.format("delta").mode("overwrite").save(f"abfss://{silver@{v_storage}}.dfs.core.windows.net/ventas_limpias")
```

Se implementaron reglas de calidad de datos (Data Quality) sobre la fuente de ventas. El filtro asegura que solo procesemos registros con un ID de venta presente y cantidades mayores a cero, evitando inconsistencias en los cálculos posteriores. Finalmente, los datos se persisten en la capa Silver bajo el formato Delta Lake.

10.3. Procesamiento de Productos y Gestión de Almacenamiento

Figura 9: Código de Procesamiento de Productos y Gestión de Almacenamiento

```
# 3. PROCESAR PRODUCTOS (JSON)

path_productos_json = f"abfss://{bronze@{v_storage}}.dfs.core.windows.net/productos.json"

# 1. Leer el JSON
df_productos = spark.read.format("json").load(path_productos_json)

# 2. Limpieza de Productos
df_productos_ok = df_productos.filter(col("id").isNotNull())

# 3. ELIMINAR LA CARPETA SILVER SI EXISTE (Esto evita el error de formato incompatible)
path_prod_silver = f"abfss://{silver@{v_storage}}.dfs.core.windows.net/productos_limpios"
dbutils.fs.rm(path_prod_silver, recurse=True)
```

Se procesa la segunda fuente en formato JSON. Un punto clave de este proceso es el uso del comando `dbutils.fs.rm`, el cual elimina de forma recursiva directorios previos en la capa Silver para evitar conflictos de metadatos o formatos incompatibles, asegurando una carga limpia de la tabla de productos.

11. Implementación: Notebook Capa_Gold_Analítica

Este notebook representa la etapa final de la arquitectura Medallion, donde se consolidan las fuentes de datos para generar reportes analíticos de alto nivel.

11.1. Configuración y Consumo de Capa Silver

Figura 10: Configuración y Consumo de Capa Silver

```
6 # 1. CONFIGURACIÓN DE PARÁMETROS Y AUTENTICACIÓN
7 v_storage = "datalaketpint1"
8 v_key = "xyyl02JGj0m0Nzsra0AXhEtVqSNnbxJTYeq/AT0eYbtHTsvnNEZp5Dj0r+IzVpM2zG9vOXGffmX7+ASTroxYSw=="
9
10 spark.conf.set(
11     f"fs.azure.account.key.{v_storage}.dfs.core.windows.net",
12     v_key
13 )
14
15 from pyspark.sql.functions import col, sum, round
16
17 # 2. DEFINICIÓN DE RUTAS
18 path_ventas_silver = f"abfss://{v_storage}.dfs.core.windows.net/ventas_limpias"
19 path_productos_silver = f"abfss://{v_storage}.dfs.core.windows.net/productos_limpios"
20 path_gold = f"abfss://{v_storage}.dfs.core.windows.net/reporte_final"
21
22 print("Cargando datos desde la capa Silver...")
23
24 # 3. LECTURA DE DATOS (DELTA FORMAT)
25 try:
26     df_ventas = spark.read.format("delta").load(path_ventas_silver)
27     df_productos = spark.read.format("delta").load(path_productos_silver)
28 except Exception as e:
29     print(f"Error al leer datos: {e}. Verificá que los Notebooks 1 y 2 hayan corrido con éxito.")
30
```

Se establecen las rutas de acceso hacia el contenedor Silver. A diferencia de los pasos anteriores, aquí ya no se leen archivos crudos (JSON/CSV), sino que se consumen tablas en formato Delta. El uso de bloques `try-except` garantiza que el pipeline sea robusto y verifique la existencia de los datos antes de proceder.

11.2 Enriquecimiento y Lógica de Negocio (Join)

Figura 11: Código de transformacion analítica y calculo de métricas

```
31 # 4. TRANSFORMACIÓN ANALÍTICA (JOIN Y ENRIQUECIMIENTO)
32 # Unimos ventas con productos para tener el nombre y el precio en una sola tabla
33 df_enriquecido = df_ventas.join(
34     df_productos,
35     df_ventas["ID_Producto"] == df_productos["id"],
36     "inner"
37 )
38
39 # 5. CÁLCULO DE MÉTRICAS (VENTA TOTAL)
40 df_final = df_enriquecido.select(
41     col("ID_Venta"),
42     col("title").alias("Producto_Nombre"),
43     col("category").alias("Categoria"),
44     col("Cantidad"),
45     col("price").alias("Precio_Unitario"),
46     (col("Cantidad") * col("price")).alias("Venta_Total")
47 )
48
```

Se realiza una de las operaciones más críticas: el Inner Join entre la tabla de ventas y la de productos. Esto permite enriquecer cada transacción con el nombre y precio unitario del producto. Se crea una nueva columna calculada Venta_Total (Cantidad * precio), aplicando redondeos para asegurar la precisión financiera del reporte final.

11.3. Persistencia en Capa Gold y Resultados Gerenciales

Figura 12: Código de capa Gold

```
49 # 6. LIMPIEZA DE DESTINO Y GUARDADO EN GOLD
50 print(f"Escribiendo resultado final en: {path_gold}")
51
52 # Borramos carpeta anterior si existe para evitar conflictos de esquema
53 dbutils.fs.rm(path_gold, recurse=True)
54
55 # Guardamos como tabla Delta
56 df_final.write.format("delta").mode("overwrite").save(path_gold)
57
58 # 7. VISUALIZACIÓN
59 print("¡PIPELINE FINALIZADO CON ÉXITO!")
60 print("Ranking de Ventas por Categoría:")
61
62 # Mostramos una agregación rápida para la visualización de Databricks
63 resumen_gerencial = df_final.groupBy("Categoria") \
64     .agg(round(sum("Venta_Total"), 2).alias("Ingresos_Totales")) \
65     .orderBy(col("Ingresos_Totales").desc())
66
67 display(resumen_gerencial)
```

Tras limpiar el directorio de destino para evitar conflictos, el DataFrame enriquecido se persiste en la capa Gold. Como resultado final, se genera un Ranking de Ventas por Categoría, que permite visualizar de forma inmediata qué sectores del catálogo están generando mayores ingresos. Esta tabla es la "Única Fuente de Verdad" para la toma de decisiones basada en datos.

12. Ejemplos de Transformación Real

Caso: Tratamiento de Nulos y Tipos de Datos

Antes (Bronze/Landing)

ID: NULL, Precio: "1500.00", Fecha: "12/03/2026"

Después (Silver/Gold)

ID: Valor por defecto asignado , Precio: 1500.00 (Float), Fecha: 2026-03-12 (ISO)

12. Desafíos, Escalabilidad y Reflexiones

12.1 Desafíos Encontrados (Troubleshooting)

Durante el desarrollo del pipeline se identificaron y resolvieron diversos desafíos técnicos relacionados con la infraestructura cloud y la integración de servicios:

Aprovisionamiento de Clusters

Se presentaron inconvenientes vinculados a la disponibilidad y cuotas de máquinas virtuales (vCPUs) en Azure Databricks dentro de la región seleccionada. El problema fue resuelto mediante la selección de un tipo de instancia compatible con los límites de capacidad de la suscripción utilizada.

Integración entre Azure Data Factory y Databricks

Fue necesario realizar configuraciones específicas en los Linked Services de Azure Data Factory para garantizar que el token de acceso personal (PAT) permitiera la ejecución automatizada de notebooks sin requerir autenticación manual durante la orquestación del pipeline.

Gestión de Formatos y Persistencia Delta

Durante las cargas de datos se detectaron conflictos de esquema al sobrescribir tablas Delta en las capas Silver y Gold. Para resolver esta situación se implementaron procesos de limpieza de directorios utilizando comandos

dbutils.fs.rm, garantizando consistencia estructural y evitando conflictos asociados a metadatos previos.

12.2 Escalabilidad y Mejoras Futuras

Con el objetivo de evolucionar esta solución hacia un entorno productivo de mayor escala y robustez, se identifican las siguientes mejoras potenciales:

Implementación de CI/CD

Integrar repositorios GitHub con Azure DevOps para automatizar el despliegue de notebooks y pipelines, permitiendo implementar prácticas de Integración y Despliegue Continuo (CI/CD).

Particionamiento y Optimización de Datos

A medida que el volumen de información aumente, se propone implementar particionamiento físico sobre tablas Delta utilizando columnas estratégicas como Fecha o Categoría, con el objetivo de optimizar tiempos de consulta y procesamiento analítico.

Monitoreo y Alertas Automatizadas

Incorporar Azure Monitor y sistemas de alertas integrados con Azure Data Factory para detectar fallos operativos de manera temprana mediante notificaciones automáticas por correo electrónico o Microsoft Teams.

Framework de Calidad de Datos

Implementar herramientas de validación como Great Expectations para automatizar controles de calidad sobre los datos de entrada, evitando que registros inconsistentes o esquemas inválidos sean promovidos hacia la capa Gold.

13. Conclusiones y Análisis de Resultados

La implementación del pipeline de datos basado en arquitectura Medallion permitió integrar, transformar y consolidar información proveniente de fuentes heterogéneas utilizando servicios de Microsoft Azure. Mediante la combinación de Azure Data Factory y Azure Databricks, se logró automatizar el flujo completo de ingesta, procesamiento y generación de datos analíticos.

La separación en capas Bronze, Silver y Gold permitió mantener trazabilidad sobre los datos originales, aplicar procesos de limpieza y validación, y generar estructuras optimizadas para análisis y reporting. Durante la etapa de transformación en Databricks se implementaron reglas de calidad de datos,

incluyendo normalización de columnas, conversión de tipos, eliminación de registros nulos y consolidación de información mediante operaciones Join.

La automatización mediante triggers programados en Azure Data Factory permitió ejecutar el pipeline de manera periódica sin intervención manual, garantizando disponibilidad diaria de la información procesada. Asimismo, el uso de Delta Lake proporcionó una estructura de almacenamiento eficiente y escalable para las capas analíticas.

Como resultado final, se obtuvo una tabla consolidada con métricas de ventas por categoría, permitiendo identificar patrones de facturación y productos con mayor impacto comercial. Esto demuestra cómo una arquitectura de datos moderna puede transformar datos crudos en información útil para la toma de decisiones.

Finalmente, el proyecto permitió aplicar conceptos fundamentales de ingeniería de datos, incluyendo orquestación de pipelines, procesamiento distribuido con PySpark, integración de múltiples formatos de datos, validación de calidad y automatización de procesos ETL en entornos cloud.